

university of groningen

Performance Engineering: Lecture #15

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-10-19

Overview



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

2D Poisson Equation: Optimizations (1/2)



- **Optimization strategy #1**: **Overlap communication and computation**
  - Steps:
    1. Initiate communication of boundary points
    2. Update interior points
    3. Wait for communications to complete
    4. Update boundary points
  - Assumptions: $\Delta t(L) = \max\left[\Delta t_{\mathrm{fp}}(L), \Delta t_{\mathrm{net}}(L)\right]$ and latency-bandwidth model holds
    - Problem becomes local performance bound for large L as

$$\Delta t \to \Delta t_{\mathrm{fp}}(L) \quad \text{for} \quad L \to \infty$$

2D Poisson Equation: Optimizations (2/2)



- **Optimization strategy #2**: **Minimise number of send-receive operations**
  - Copy halo into single buffer before communication (manual data packing)
  - Alternative solution: Use derived data types
- Performance benefits difficult to predict as message rates improved
  - But: Higher network bandwidth demands due to additional overhead (message headers, minimal message length)
- **Optimization strategy #3**: **Use Cartesian process topology**

Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

Introduction



- Optimization and performance modelling strategies
  - Assess complexity of collective communication operations
  - Reduce number of collective communication operations
  - Avoid single processes being delayed

Complexity of Selected Collectives



- **Barrier** and **reduce**
  - Typical implementation: Binary tree
  - **Complexity**: $\log_2 P$
- **Gather** or **scatter**
  - One process receives (sends) data from (to) $P - 1$ other nodes
  - **Complexity**: P

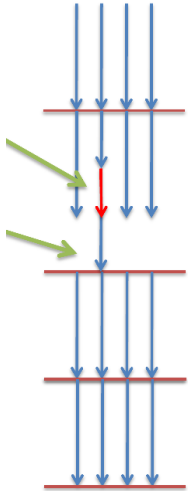
☞ **Collectives limit scalability**

- **Possible mitigation strategy**: Reduce number of collectives
  - Synchronisation only where strictly necessary
  - Data packing

OS Noise: Barriers

- **Problem**: A single delayed process makes all other processes wait
  - Example: barrier
- **Possible root cause**: OS delays execution of individual processes
  - Example: Daemons, interrupts, etc.

☞ **OS noise**

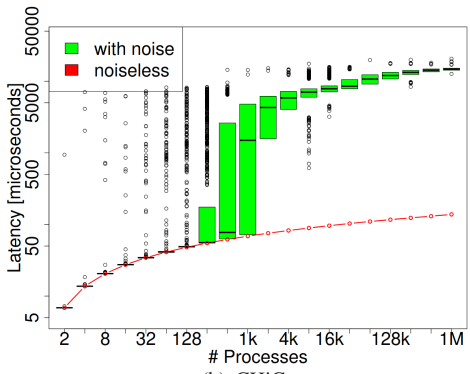


[Hoefler et al.; 2012]

OS Noise: Allreduce

- Analysis of `MPI_Allreduce` on a cluster lacking HPC optimizations at OS-level

[Hoefler et al.; 2010]



(b) CHiC

Content



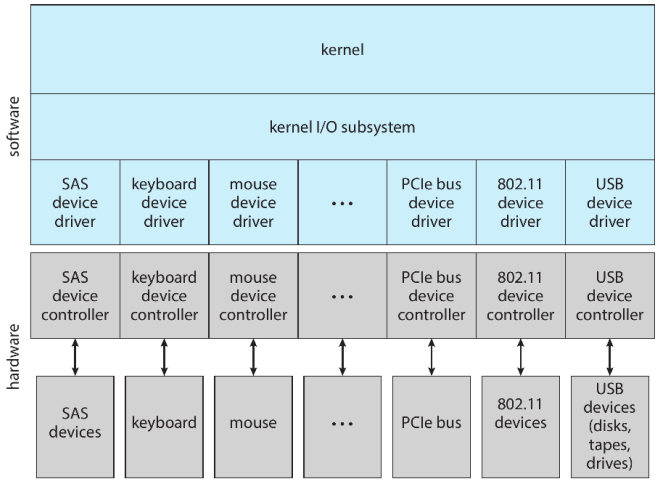
1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. **Storage Software: OS**
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

I/O Interface: OS Perspective



- **OS perspective**:
  - The operating system (OS) provides an **interface layer** between **applications** and underlying **I/O hardware**
  - I/O performance characteristics depend crucially on the OS
- **Approach** to interface layer design
  - **Generic classes** of I/O system calls to access devices with common behaviours
  - **Layered design** hides differences among I/O controllers from higher layers in the kernel
- **Challenges**
  - Each OS has its own I/O subsystem structures and device driver framework
    - Focus here: Linux

A Kernel I/O Structure



I/O Devices Types



- Character devices
- **Block devices** ⇐ **Most relevant for storage**
- Network devices

Block Devices



- **Block device** = An I/O device that is **randomly accessible** with **block-size chunks** being the smallest I/O unit
- By using block sizes $\gg 1\,\mathrm{B}$ makes this I/O device suitable if larger amounts of data are involved
- **Examples**: hard disk drives (HDD), solid-state drives (SSD)
- **Interface semantics**: read, write, seek

Kernel I/O Scheduling



- I/O requests are put in a **wait queue**
- The kernel is responsible for **request scheduling**
  - Scheduling means determining a **good request order** for **I/O request execution**
- Scheduling criteria
  - Improve overall system performance
  - Ensure fairness
  - Reduce average waiting times
  - Enforce quality of service (QoS)
- Tuning the scheduler for performance is beyond the scope of this lecture

I/O Buffering



- **Buffer** = Memory area used to store data being transferred
  - Transfer between 2 devices or between application and device
- What is the benefit of buffering? Why is buffering required?
  - Cope with **speed-mismatch** between data producer and consumer
  - Support adaptions required in case of **data transfer sizes discrepancies**
  - Optimise **blocking calls**
    - Example: Copy application data first to memory, return, and write data asynchronously to disk
- The OS typically provides means for I/O buffering
- **Buffering can be part of an performance optimization strategy**

I/O Caching



- **Cache** = Hardware or software component that transparently manages a copy of the data for faster access
  - **Transparent** means
    - The accessor observes the same functional behaviour as for accessing the original data
    - The cache manages coherence between original data and copies in the cache
- Important features for caches being beneficial
  - Access to the cache is much faster
  - Spatial and temporal localities make reuse of the cached data likely
- **Caching can be part of an performance optimization strategy**

Content

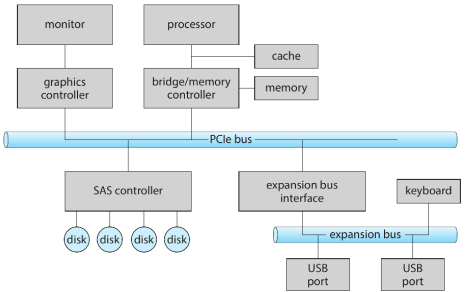


1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. **Storage Hardware**
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

I/O System Example



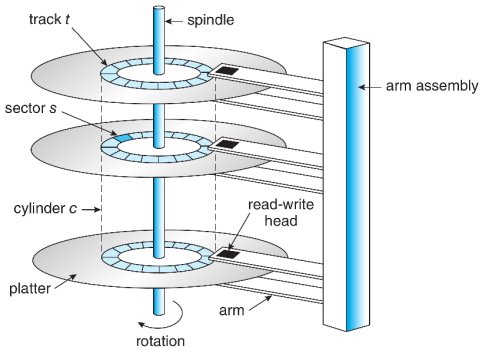
- Computer system with I/O devices connected via an **PCIe bus**
  - **PCI Express (Peripheral Component Interconnect Express)** is a high-speed serial computer expansion bus standard
  - Current generation **PCIe 5.0** supports up to **32 GT/s per lane**
  - Note: At the physical level, PCIe is based on point-to-point connections
- **Storage devices** can be attached through some intermediate controllers
  - Example: SAS controller



Storage Devices: Hard Disk Drives (HDD) (1/2)



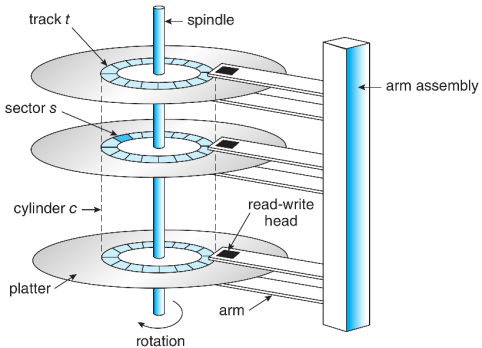
- Hardware organization
  - **Disk platter**: Flat circular shape covered with a magnetic material
  - **Heads** attached to a reading arm
  - Organization of the platters: Circular **tracks** subdivided in **sectors**
  - **Cylinder**: Set of tracks at the same position but different platters



Storage Devices: Hard Disk Drives (HDD) (2/2)



- Typical performance features
  - **Seek time**: Time to position head
    - Up to $10\,\text{ms}$
  - **Rotational latency**: Time for target sector to appear at the head
    - About $2\,\text{ms}$ to $5\,\text{ms}$
  - **Data rate**: Data transfer rate
    - $O(100)\,\text{MB}\,\text{s}^{-1}$



Storage Devices: Non-Volatile Memory (NVM)

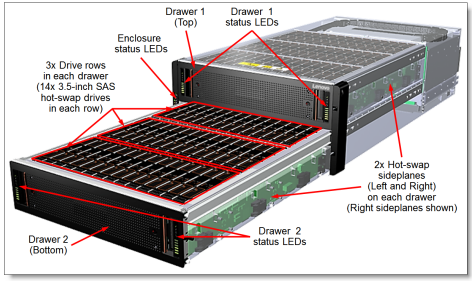


- Storage devices based on non-volatile memory technologies
  - 3D XPoint (discontinued)
  - NAND Flash
    - NAND Flash is organised in **blocks** comprising multiple **pages**
    - Pages cannot be overwritten, have to first erase blocks
      - To avoid immediate erasure, pages are marked as invalid
- **Advantages**:
  - Very high sequential access bandwidth ($> 5\,\mathrm{GB\,s^{-1}}$ for read)
  - Low access latencies and high bandwidth for random access
  - Low power at rest
- **Disadvantages**:
  - Less capacity per drive
  - Limited endurance

Storage Devices: JBOD

- **JBOD** = Just a Bunch of Disks is a collection of disks in a computer system combined as one logical volume
- Design goals
  - High storage density
  - Reasonable large aggregate bandwidth

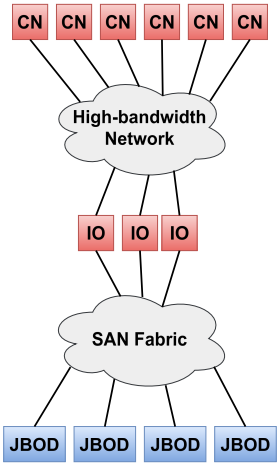
Example: Expansion enclosure, up to 84 drives, $3\times$ 12 Gb SAS $\times 4$



[Lenovo; 2025]

Network-attached, Parallel Storage

- Storage is often **network-attached**
- Typical HPC design: Compute cluster + storage cluster
  - Set of compute nodes connected to a set of storage nodes
  - Storage nodes connected to a set of JBODs
  - Optional: Tape backend



Storage Technologies: RAID



- **RAID** = Redundant Array of Independent Disks
  - Multiple physical disks are combined into a single unit to form one logical disk
  - Interface typically implemented by a controller in hardware
- **Advantages** of using RAID
  - **Protection against data-loss** due to redundant copy of data
  - **Performance increase** by spreading data over multiple disks (**striping**)
- Different RAID schemes ("levels"): RAID0, ..., RAID6

Level	$\min(n)$	Space efficiency	Fault tolerance
RAID1	2	$1/n$	$n-1$
RAID5	3	$1-1/n$	1
RAID6	4	$1-2/n$	2

n ... number of disks

Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. **Storage Software: File Systems (and others)**
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

File Concept



- **File** = A named collection of related information
  - An **abstraction** that provides a way for storing and retrieving information
  - The size of a file is not assumed to be fixed
- User perspective: **Smallest allotment of storage**
  - Files are typically stored on a storage device
  - Files may be created in memory

File System (1/2)



- **File system** = A file system is a technology that an operating system uses to manage files and the available storage resources
- A file system bridges the **logical structure** exposed to the user and the **physical structure** of the hardware
- File systems have been primarily designed to enable the use of non-volatile storage devices
- Key hardware features of such devices
  - Data can be (logically) **replaced in place**
  - **Random accessibility**
  - Transfer between storage and memory in units of **blocks**
    - Block sizes typically in the range between $512\,\mathrm{B}$ to $4096\,\mathrm{B}$

File System (2/2)



- Performance-critical aspects
  - **Metadata**: How to store information about the file
  - **Efficiency**: How to
    - Use the available storage space efficiently
    - Ensure that reading and writing data is as fast as possible
- File systems have been optimized wrt the performance characteristics of available hardware

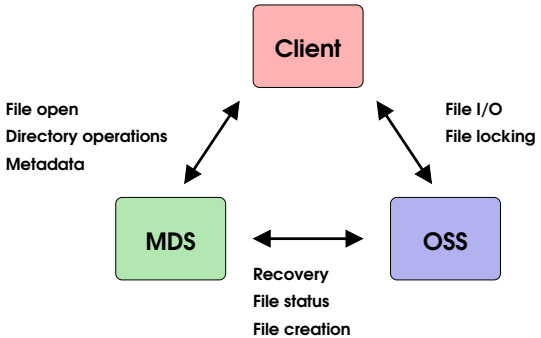
Parallel File Systems: Lustre (1/2)



- To reach very high performance and large capacity, parallel storage architectures based on various network-attached I/O servers are needed
  - Widely used example: **Lustre**
- Lustre features
  - POSIX compliant
  - Scales to a large number of clients
    - Current practical range: 100-100000
  - File system scalability
    - Current practical range: up to 512 PBytes, 4 billion files
  - Claims to have no single point of failure
  - Support of multiple network protocols

Parallel File Systems: Lustre (2/2)

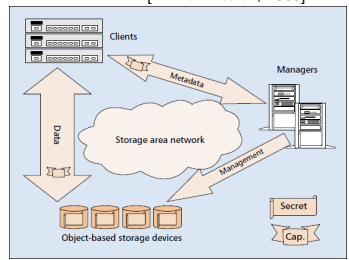
- Key components:
  - **Metadata Server (MDS)**: Server which manages the names and directories in the file system
  - **Object Storage Servers (OSS)**: Server which provides access to the storage
  - **Client**: Software running on compute nodes that mount the Lustre file system



Object Stores

- **Storage abstraction**: Data is exposed and managed as objects
  - Data is not managed as files or blocks
  - Lower layers of the storage architecture likely still manage data as blocks
  - A file system may be deployed on top of an object store
- Objects are typically organised in buckets
  - Unique bucket identifier (if applicable)
  - Object identifier unique within a bucket or across the full system
- **Assumed usage**: Single client writes object once, many clients cause many reads
- Popular **object store solution**: **Ceph**

[E. Riedel et al., 2003]



Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

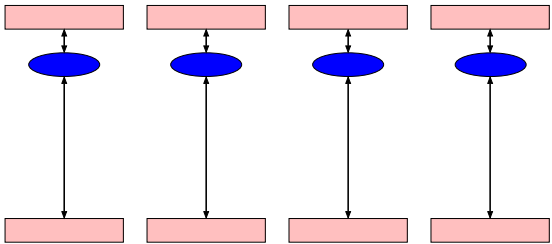
POSIX



- **POSIX file system** = A file system that provides the standard set of file operations, metadata, and behavioral semantics defined by the POSIX standard
  - POSIX = Portable Operating System Interface
  - First version of the **POSIX standard** released by IEEE in 1988
- **Key features**
  - Streaming I/O semantics (write to or read from data streams)
  - Synchronous I/O
- **Shortcomings** from a performance engineering perspective
  - Strict consistency model affects performance
  - High abstraction level makes it agnostic of the underlying hardware
  - Lacking support of asynchronous and parallel I/O
  - Metadata handling can become challenging

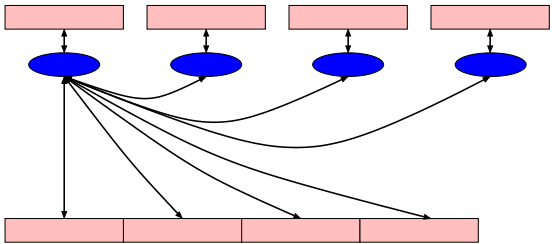
Digression: Parallel Case / Independent I/O

- **Advantage(s)**: Highly parallel without dependencies
- **Disadvantage(s)**: Many small files
  - Metadata performance bottleneck, inode limits in file system



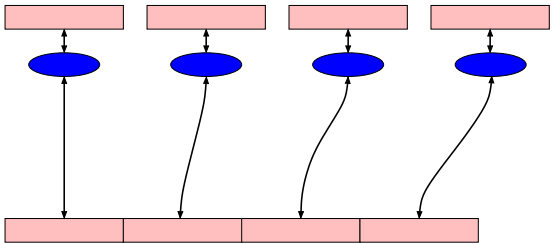
Digression: Parallel Case / Single Rank

- **Advantage(s)**:
  - Easy to implement
  - Single file
- **Disadvantage(s)**:
  - All I/O routed through a single node
  - Non-scalable approach



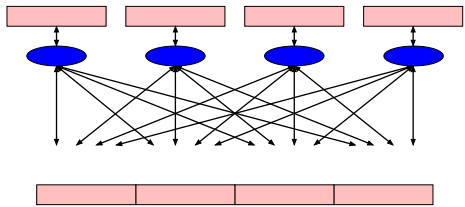
Digression: Parallel Case / Single File

- **Advantage(s)**:
  - Highly parallel
  - Single file
- **Disadvantage(s)**: High risk of concurrent storage block update
  - Possibly high costs of file block locking due to lock contention



MPI I/O

- MPI provides useful features for optimising parallel I/O
  - Non-blocking communication operations
  - Collective communication operations
  - Communicators to separate application-level message passing from I/O-related data transport
  - Support for defining application-specific data types
- **MPI I/O** uses MPI as a data transport engine to move data efficiently from or to storage
- **Performance engineering opportunity**: Cooperative I/O



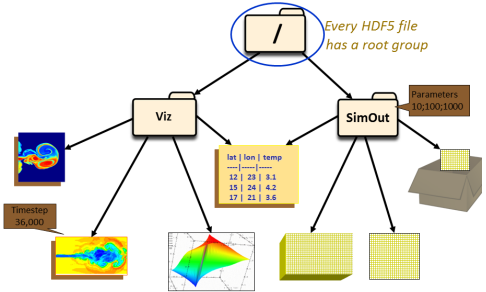
Digression: Container File Format



- **Container file format** = File format that supports combining multiple digital resources into a single digital resource
- Popular container file formats
  - Zip
  - tar
  - cpio
- Design approaches
  - Sequence of records with header
  - Optional: central directory
- **Performance engineering opportunity**: Reduce metadata operations

HDF5

- Example container file format: **HDF**
  - HDF = Hierarchical Data Format
  - A technology suite for managing (large) data collections
- HDF5 components:
  - Data model
  - API + software library
  - File format
- Features:
  - Various language bindings:
    C, Fortran, Java, C++, Python
  - Parallel I/O support based on MPI
  - Integrated support for compression



[HD5 Group]

Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

IO500 List



- The **IO500 List** shows HPC storage systems ranked according to their IO500 benchmark results
  - `https://io500.org/`
  - List is published $2\times$ per year
- The **IO500 Benchmark** is a set of workloads based on following benchmarks and tools:
  - IOR
  - mdtest
  - find

IOR Benchmark



- Benchmark designed to mimic I/O data patterns of HPC applications [Shan, Shalf; 2007]
  - Documentation: `https://ior.readthedocs.io/en/latest/`
- Features
  - Flexible support of various, regular, parallel I/O patterns
  - Support of various I/O interfaces: POSIX, MPI I/O, HDF5
  - Support of file system specific configurations (e.g. Lustre)

mdtest Benchmark

- Benchmark designed to stress file system's metadata operations
  - The benchmark has been merged with IOR

```
-- started at 10/19/2025 11:05:51 --

mdtest-1.9.3 was launched with 1 total task(s) on 1 node(s)
Command line used: ../git/mdtest -i 100 -n 1000
Path: /home/pleiter/hpc/prj/io/benchmarks/mdtest
FS: 787.1 GiB   Used FS: 86.2%   Inodes: 50.1 Mi   Used Inodes: 6.1%

1 tasks, 1000 files/directories

SUMMARY: (of 100 iterations)
   Operation                     Max            Min           Mean        Std Dev
   ---------                     ---            ---           ----        -------
   Directory creation:      63696.888      13284.010      43860.525      10303.765
   Directory stat    :     804836.747     377843.413     659962.999      66531.147
   Directory removal :     117406.927      59016.419      96714.344       9697.831
   File creation     :     109980.995      61351.097      90665.627       8066.919
   File stat         :     788692.671     377118.842     653536.902      70682.924
   File read         :     595248.016     327637.787     486837.878      47124.698
   File removal      :     210986.177      98456.827     171803.624      20928.307
   Tree creation     :      84210.526      15029.684      49078.978      20963.585
   Tree removal      :       2052.301        633.642       1533.078        235.798

-- finished at 10/19/2025 11:05:57 --
```

IO500: IOEasy



- IOR benchmark using well optimized I/O patterns
  - Sequential read/write
  - Each process creates it's own file
  - Use of large records: `transferSize`$=2$ MiB
  - Large file size: `blockSize`$=9\,920\,000$ MiB
- Select results for EuroHPC pre-exascale system **Leonardo** [IO500; 2025]
  - Lustre based on DDN's Exascaler

Workload	Result
Easy write	1552 GiB/s
Easy read	1886 GiB/s

IO500: IOHard



- IOR benchmark mimicking applications that require a random workload
  - Random read/write
  - Shared file
  - Use of smaller records: `transferSize`$=47\,008\,\mathrm{B}$

Question: What could be the motivation for this choice of `transferSize`?

IO500: IOHard



- IOR benchmark mimicking applications that require a random workload
  - Random read/write
  - Shared file
  - Use of smaller records: `transferSize`$=47\,008\,\mathrm{B}$

Question: What could be the motivation for this choice of `transferSize`?

This value does not match typical file system and hardware block sizes

IO500: IOHard



- IOR benchmark mimicking applications that require a random workload
  - Random read/write
  - Shared file
  - Use of smaller records: `transferSize`$=47\,008\,\mathrm{B}$

Question: What could be the motivation for this choice of `transferSize`?

This value does not match typical file system and hardware block sizes

- Select results for EuroHPC pre-exascale system **Leonardo** [IO500; 2025]
  - Lustre based on DDN's Exascaler

Workload	Result
Hard write	$155\,\mathrm{GiB/s}$
Hard read	$933\,\mathrm{GiB/s}$

  - Note: $10\times$ and $2\times$ worse compared to IOEasy

IO500: MDEasy



- mdtest benchmark operating on small objects
- Select results for EuroHPC pre-exascale system **Leonardo** [IO500; 2025]
  - Lustre based on DDN's Exascaler

Workload	Result
Easy write	$397\,\mathrm{kIOP/s}$
Easy stat	$807\,\mathrm{kIOP/s}$
Easy delete	$335\,\mathrm{kIOP/s}$

Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. Final

iostat

- Standard Linux tool that provides input/output statistics for devices and partitions
- Good for monitoring while I/O is ongoing

```
1  Linux 6.8.0-85-generic (zam344)          19/10/25        _x86_64_        (8 CPU)
2
3  avg-cpu:  %user   %nice %system %iowait  %steal   %idle
4             6.37    0.06   10.65    0.29    0.00   82.63
5
6  Device            r/s     rkB/s   rrqm/s  %rrqm r_await rareq-sz     w/s     wkB/s   wrqm/s  %wrqm w_await wareq-sz     d/s     dkB/s   drqm/s  %drqm d_await dareq-sz     f/s f_await  aqu-sz  %util
7  nvme0n1          7.38    191.84     3.02  29.00    0.21    25.99    7.29    248.45     9.98  57.81    2.94    34.10    0.00     0.00     0.00   0.00    0.00     0.00    0.59    0.72    0.02   0.52
8
9
10 avg-cpu:  %user   %nice %system %iowait  %steal   %idle
11            3.77    0.00   23.30    1.01    0.00   71.92
12
13 Device            r/s     rkB/s   rrqm/s  %rrqm r_await rareq-sz     w/s     wkB/s   wrqm/s  %wrqm w_await wareq-sz     d/s     dkB/s   drqm/s  %drqm d_await dareq-sz     f/s f_await  aqu-sz  %util
14 nvme0n1         67.50    270.00     0.00   0.00    0.49     4.00 3003.50 1254186.00    60.00   1.96    3.74  417.57    0.00     0.00     0.00   0.00    0.00     0.00    0.00    0.00   11.28  35.60
```

- Similar other tools: `iotop`, `sysstat`

Linux perf



- Trace all I/O-related system calls for a specific command

```
sudo perf trace -e read,write dd if=/dev/zero of=/dev/null count=5
```

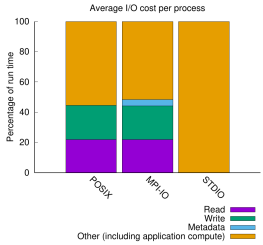
- Collect aggregate information using the Linux Kernel Tracepoint API:

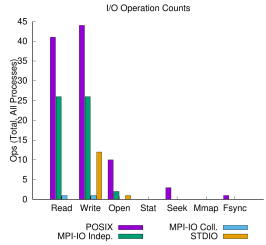
```
sudo perf trace --duration 5 \
     -e block:block_rq_issue,block:block_rq_complete \
     dd if=/dev/zero of=/dev/null count=5
```

  - `block_rq_issue`: traces I/O requests that have been issued
  - `block_rq_complete`: traces I/O requests that have completed

Darshan

- Darshan is an HPC I/O characterisation tool
  - Documentation: `https://www.mcs.anl.gov/research/projects/darshan/`
- Profiling approach: Intercept I/O calls by pre-loading a shared library
  - No recompilation is required
  - Intercepts the most relevant POSIX and MPI I/O functions
- Various tools available to process profiling files





Content



1. Case Study: Halo Update (2)
2. Collective Communication Optimization
3. Storage Software: OS
4. Storage Hardware
5. Storage Software: File Systems (and others)
6. I/O APIs
7. I/O Performance Benchmarks
8. I/O Performance Monitoring
9. **Final**

Summary Lecture #15



- Collective communication optimization
  - Data packing, use of MPI derived data types
  - Impact of single process delays
- Background on storage architectures
  - OS-level software
  - Hardware
  - I/O APIs
- I/O benchmarks and monitoring tools
- I/O performance challenges
  - Expensive metadata operations
  - Low I/O bandwidth

Questions?



[Wiki Media]